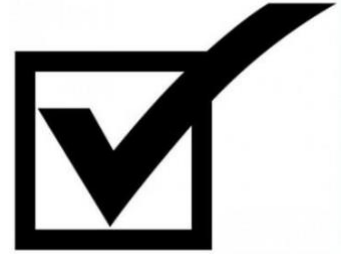




**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



C++ PROGRAMMING

(335)

STATE 2022

APPLICATION KNOWLEDGE:

GPS Distance Calculation Utility _____ (400 points)

TOTAL POINTS _____ (***400 points***)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

GPS Distance Calculation Utility

In this exercise, you will create a C++ console application that shows the closest GPS coordinates to a current position. Such a utility could be used for an application that finds the closest gas station to your current position.

On application start, the file will be loaded. The first line of the file is the current position, and each subsequent line is a target position. These records are space separated and show latitude first followed by longitude.

```
Haversine Distance to 37.5198, 176.545 from 11.1057, -148.587 : 4547.62km
Haversine Distance to 62.5637, -120.411 from 11.1057, -148.587 : 6147.31km
Haversine Distance to -2.12143, 42.4831 from 11.1057, -148.587 : 18437.1km
Haversine Distance to -55.7161, -103.68 from 11.1057, -148.587 : 8513.74km
Haversine Distance to -9.42737, 45.0264 from 11.1057, -148.587 : 18514.1km

Closest to furthest...
37.5198, 176.545 : 4547.62km
62.5637, -120.411 : 6147.31km
-55.7161, -103.68 : 8513.74km
-2.12143, 42.4831 : 18437.1km
-9.42737, 45.0264 : 18514.1km
Press any key to continue . . .
```

Figure 1: Expected output, showing distance to each set of coordinates and the sorted results

To calculate distance between to GPS points, the Haversine formula can be used.

$$d = 2r * \arcsin \arcsin \left(\sqrt{\sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos \cos (\varphi_1) \cos \cos (\varphi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

Where λ is longitude, φ is latitude, and r is the radius of the Earth (6371km). Note that these longitude and latitude values are expected in radians, not degrees (which is the format stored in the file). Make use of the math functions in C++ to simplify these operations.

$$rad = deg * \frac{\pi}{180.0}$$

Where π is defined as 3.14159.

Requirements:

1. You must create a C++ console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. When application starts, application shall load data from included “coordinates_data.txt” file. Each line in the file is a new coordinate, where a line contains the latitude followed by the longitude. The first coordinate in the file is the current position, all subsequent coordinates are target positions.
 - a. If the file was not able to be opened, application shall show an error message indicating that there was an error.
 - b. If the data structure of choice is empty after the file is loaded, application shall display an error message that there were no data elements found.
4. Once all data has been loaded and those checks are complete, the application shall calculate and report all distances as shown in Figure 1. The width of each data field shall be set to 8.
5. Once distances to each position has been calculated, those GPS locations will be sorted by their distance in order of closest to furthest and then reported as shown in Figure 1.
6. All coordinates shall be displayed in the form of {latitude}, {longitude}.
7. A class, GPS, shall be created for housing the information of a single GPS location.
 - a. Appropriate getters and setters shall be created for the longitude and latitude values.
 - b. A public accessor for distance shall be created to perform the Haversine calculation.
 - c. A constructor that accepts an ifstream object shall be created to parse a data line.
 - d. A display method shall be created for abstracting the reporting of a GPS location.
8. The sorted list of GPS locations shall be output into a created “distances.txt” file prior to application completion.

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____ 10 pts
The project is named according to the naming conventions	_____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB	_____ 10 pts
--	--------------

If the program does not execute, then the remaining items in this section receive a score of zero.

Application displays error message if cannot load file	_____ 20 pts
Application properly loads data file	_____ 20 pts
Application properly reports correct distances	_____ 40 pts
Application properly reports correctly sorted coordinates	_____ 40 pts
Application properly handles error conditions	_____ 20 pts
Application properly saves sorted coordinates into file	_____ 30 pts

Source Code Review

Code is commented at the top, for each function, and as needed	_____ 20 pts
Code uses reasonable and consistent variable naming conventions	_____ 20 pts
The specified functions are present and are constructed in a logical manner	_____ 50 pts
The specified classes are present and are constructed in a logical manner	_____ 50 pts
File error handling is performed properly	_____ 30 pts
Main function performs only reporting functionality	_____ 30 pts

Total Points: 400 pts

Source Code:

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <iomanip>
#include "GPS.h"

using namespace std;

const string inputFilePath = "coordinates_data.txt";
const string outputFilePath = "distances.txt";

void sortDistances(vector<GPS*>& locs, GPS* curLoc)
{
    for (int i = 0; i < locs.size() - 1; i++)
    {
        if (curLoc->GetDistance(locs[i]) > curLoc->GetDistance(locs[i + 1]))
        {
            std::swap(locs[i], locs[i + 1]);
        }
    }
}

int main()
{
    ifstream inputFile(inputFilePath);
    ofstream outputFile(outputFilePath);
    if (inputFile.is_open())
    {
        vector<GPS*> locs = vector<GPS*>();
        GPS* curLoc = new GPS(inputFile);

        while (!inputFile.eof())
        {
            GPS* tgtLoc = new GPS(inputFile);

            cout << "Haversine Distance to ";
            tgtLoc->Display();
            cout << " from ";
            curLoc->Display();
            cout << " : " << setw(W) << curLoc->GetDistance(tgtLoc) << "km" <<

endl;

            locs.push_back(tgtLoc);
        }

        sortDistances(locs, curLoc);

        cout << endl << "Closest to furthest..." << endl;

        outputFile << curLoc->GetLatitude() << "," << curLoc->GetLongitude() <<

endl;

        for (int i = 0; i < locs.size(); i++)
        {
            locs[i]->Display();
```

```
        cout << " : " << setw(W) << curLoc->GetDistance(locs[i]) << "km" << endl;

        outputFile << locs[i]->GetLatitude() << "," << locs[i]->GetLongitude() << "," << curLoc->GetDistance(locs[i]) << endl;
    }
    else
    {
        cout << "Error reading file." << endl;
    }

    inputFile.close();
    outputFile.close();

    system("pause");
    return 0;
}
```

```
#pragma once

#include <math.h>
#include <fstream>
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

#define M_PI 3.14159265358979323846
#define R 6371
#define W 8

class GPS
{
public:
    GPS(ifstream& stream);
    double GetDistance(GPS* other);

    void SetLongitude(double longitude);
    void SetLatitude(double latitude);
    double GetLongitude();
    double GetLatitude();

    void Display();

private:
    double longitude;
    double latitude;
};
```



```

#pragma once

#include "GPS.h"

void GPS::SetLatitude(double latitude)
{
    this->latitude = latitude;
}

void GPS::SetLongitude(double longitude)
{
    this->longitude = longitude;
}

double GPS::GetLatitude()
{
    return this->latitude;
}

double GPS::GetLongitude()
{
    return this->longitude;
}

GPS::GPS(ifstream& stream)
{
    double lat, lng = 0;
    stream >> lat >> lng;
    SetLatitude(lat);
    SetLongitude(lng);
}

void GPS::Display()
{
    cout << setw(W) << this->GetLatitude() << ", " << setw(W) << this->GetLongitude();
}

double GPS::GetDistance(GPS* other)
{
    // Calculate longitude/latitude differences
    double latDif = (other->GetLatitude() - this->GetLatitude()) * M_PI / 180.0;
    double lonDif = (other->GetLongitude() - this->GetLongitude()) * M_PI / 180.0;

    // Convert latitude to radians
    double radLat1 = (this->GetLatitude() * M_PI) / 180.0;
    double radLat2 = (other->GetLatitude() * M_PI) / 180.0;

    // Perform Haversine distance calculation
    double sineA = pow(sin(latDif / 2), 2) + pow(sin(lonDif / 2), 2) * cos(radLat1) *
cos(radLat2);
    double sineC = 2 * asin(sqrt(sineA));

    return R * sineC;
}

```